NTNU
Kunnskap for en bedre verden

Department of Engineering Cybernetics

TTK4235 - Embedded Systems

Elevator Project Report

Author:
Benjaminas Visockis

GitHub Repository:
https://github.com/theBadMusician/TTK4235-elevator-project/

Codebase Documentation:
https://thebadmusician.github.io/TTK4235-elevator-project/

23rd March 2026

Table of Contents

List of Figures i

List of Tables ii

List of Acronyms ii

1 Introduction 1

2 Methods 1

2.1 Product Requirement Specifications 1

2.2 Software Architecture 1

2.2.1 Module Design 1

2.2.2 System Integration & Data Flow 1

2.2.3 Finite State Machine Architecture 2

2.3 Software Implementation & Development 4

2.3.1 Development Toolchain 4

2.3.2 Version Control & Continuous Integration 5

3 Results 5

3.1 System Performance and Validation 5

3.2 Factory Acceptance Test 5

4 Discussion 8

4.1 V-model & Iterative Development Process 8

4.2 Robustness, Scalability, and Maintainability 8

4.3 AI Disclosure 9

5 Conclusion 9

Bibliography 9

Appendix 10

A Product Requirements Document (PRD) 10

List of Figures

1 Unified Modelling Language (UML) module dependency diagram illustrating the dependencies and interactions within the elevator system. 2

2 UML state machine diagram of the elevator system. 3

3 UML state machine diagram for the internal movement logic of the elevator system. 4
4 Initial software architecture concept with a Scheduler module. 8

List of Tables

1 Verification of Startup & Initialization Requirements 6
2 Verification of Order Handling & Routing Logic 6
3 Verification of Hardware Input/Output (I/O) and Indicators 6
4 Verification of Door Operations & Timers 7
5 Verification of Safety Constraints & Emergency Stop 7
6 Verification of System Robustness & Recovery 7

List of Acronyms

API	Application Programming Interface
CI	Continuous Integration
CPU	Central Processing Unit
FAT	Factory Acceptance Test
FSM	Finite State Machine
GCC	GNU Compiler Collection
HAL	Hardware Abstraction Layer
HMI	Human-Machine Interface
HTML	HyperText Markup Language
I/O	Input/Output
PRD	Product Requirements Document
PRS	Product Requirements Specification
SOC	Separation of Concerns
UML	Unified Modelling Language

1 Introduction

This project involves the development of a real-time control system for a physical elevator model. The software is implemented in C and interfaces with the hardware via an Arduino microcontroller. The primary objective is to create a robust, event-driven system capable of safely managing concurrent user requests across multiple floors while adhering to real-time performance constraints.

To ensure a structured development lifecycle and verification of the system's acceptance criteria, the project followed the pragmatic V-model. This methodology, combined with Unified Modelling Language (UML) design principles, allowed the complex control logic to be decomposed into independent modules, centered around Finite State Machine (FSM) and an elevator-routing logic.

2 Methods

2.1 Product Requirement Specifications

The Product Requirements Document (PRD), provided in Appendix B of the project manual (ITK 2026), served as the basis for the system's concept design, module outline, and software development. The functionality and testing specifications from the PRD are attached in appendix A.

2.2 Software Architecture

2.2.1 Module Design

By analyzing the PRD and test specifications in appendix A, the system architecture was decomposed into distinct, decoupled modules to strictly adhere to the principle of Separation of Concerns (SOC). This "divide and conquer strategy" is widely used to design flexible and modularized software (Laplante and Kassab 2023, p. 115). The final implemented software architecture contains the following independent modules:

- **FSM (Finite State Machine):** Acts as the central control unit, governing the high-level behaviour of the elevator. It coordinates transitions between operational states (e.g., IDLE, MOVING, DOOR_OPEN) based on hardware inputs and queue evaluations.
- **Movement:** Implements the core elevator routing logic. It is responsible for evaluating the active orders to determine the optimal motor direction and dictating when the elevator must halt to service a requested floor.
- **HMI (Human-Machine Interface):** Manages all direct interactions with the physical hardware. It continuously polls floor sensors, detects button presses with debouncing logic, and updates the corresponding physical indicator lamps on the panel.
- **Timer:** Provides non-blocking time-keeping. Timer module manages asynchronous delays for system events, such as the required door-open duration and button debounce interval, ensuring the main execution loop is never halted. The module provides a simple method for blocking delays, as well.
- **Utils:** Houses utility functions for array inspection, such as determining if an array, e.g., the order queue, is empty.

2.2.2 System Integration & Data Flow

The entry point of the application (`main.c`) acts solely as a clock generator. It initiates an infinite loop that calls the central state machine (`fsm_spin`) and then yields the thread using nanosleep. This creates a deterministic, non-blocking polling rate of 50 Hz, ensuring the Central Processing Unit (CPU) is not bottlenecked while waiting for hardware Input/Output (I/O).

The FSM module serves as the central brain of the system. To prevent conflicting hardware commands, it is the only module that dictates high-level state changes, which is further elaborated

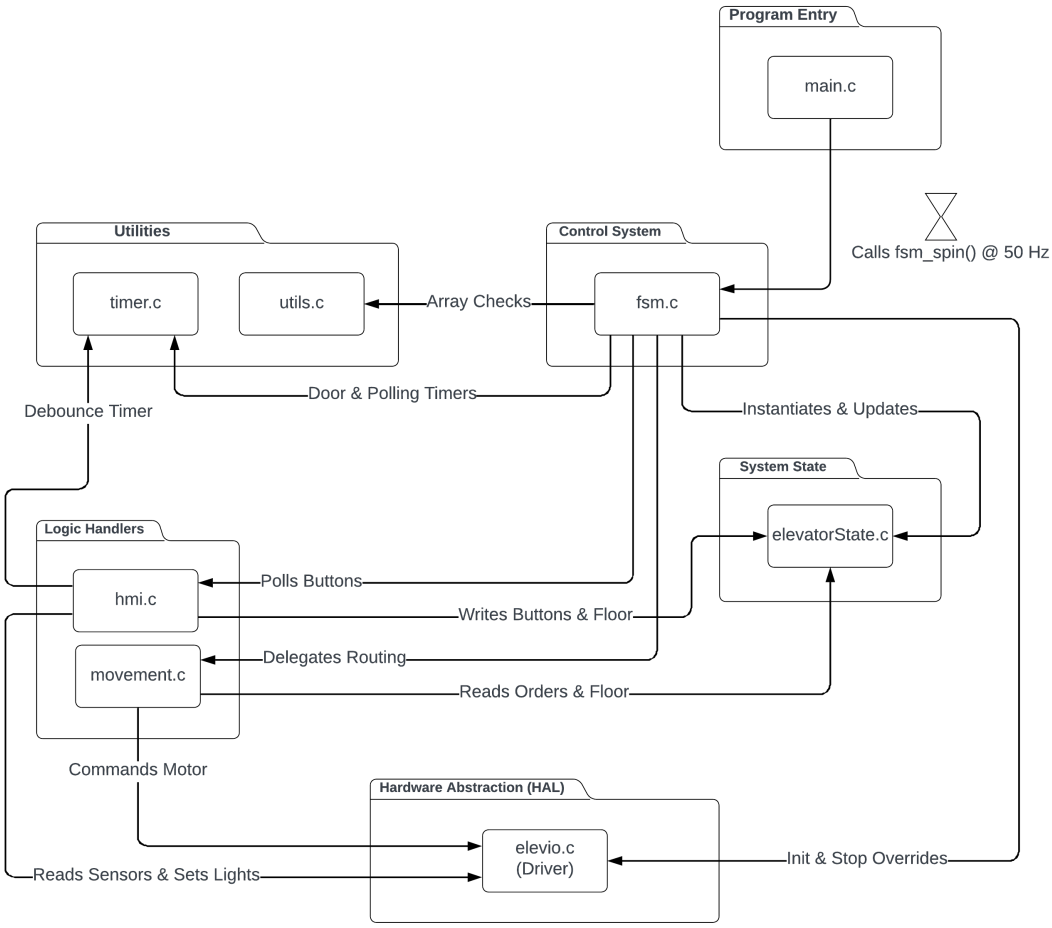

Figure 1: UML module dependency diagram illustrating the dependencies and interactions within the elevator system.

in section 2.2.3. Data flows through a centralized shared state (`elevatorState`). Hardware inputs are polled by the Human-Machine Interface (HMI) and written to this state. The routing logic (in `movement.c`) then reads this sanitized state data to calculate the optimal motor direction, isolated from the hardware driver (`elevio.c`).

Direct interaction with the elevator hardware is contained in the Hardware Abstraction Layer (HAL). The elevio driver is wrapped by the HMI and movement handlers. This abstraction layer ensures that the core FSM and routing logic remain mostly platform-independent, allowing for easier maintenance and potential future adaptations to different hardware configurations. There are a few exceptions to this, such as motor stop functionality, which is often triggered from within states to ensure quick elevator halt. However, the core movement logic is entirely decoupled from the hardware driver, allowing for potential reuse in different contexts or with different hardware interfaces.

2.2.3 Finite State Machine Architecture

The core control logic of the elevator is implemented as an event-driven FSM. This architecture manages the high-level operation of the elevator, dictating how the system responds to hardware inputs such as sensor triggers, button presses, order requests, and internal timers. The system's behaviour is documented using standard UML State Machine Diagrams.

The FSM consists of five distinct states: INIT, IDLE, MOVING, DOOR_OPEN, and STOP, visualized in figure 2. The system utilizes transition guards to ensure safe operation. For instance,

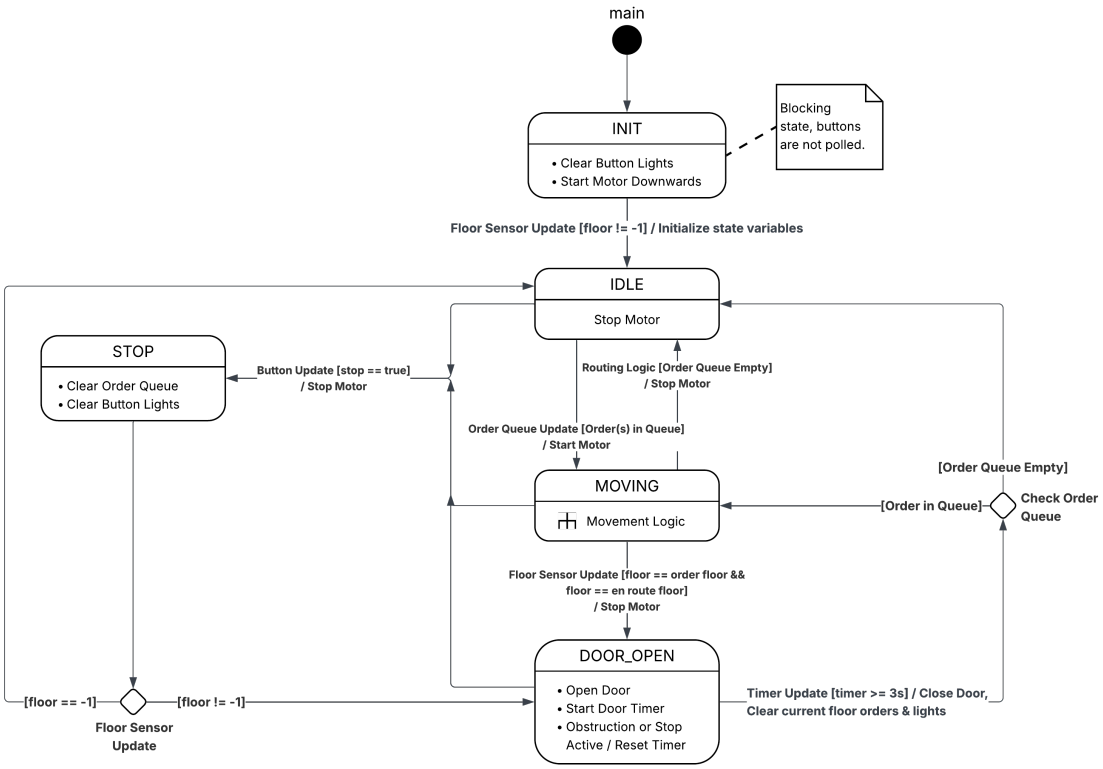

Figure 2: UML state machine diagram of the elevator system.

the transition from DOOR_OPEN back to standard operation requires a 3-second timer expiration per specifications in the PRD, alongside the immediate clearing of the current floor's order queue and indicator lights. A critical safety feature is the emergency stop functionality, modeled as an interrupting state. Transition paths from the active operational states (IDLE, MOVING, DOOR_OPEN) are routed via a shared line to the STOP state to reduce clutter.

To separate the high-level system operation from the low(er)-level movement logic, the MOVING state is implemented as an UML subactivity, denoted by the rake icon (Fowler 2004, pp. 91–93). This diagram design reflects the SOC in the module design (section 2.2.1), where the FSM is separated from routing, stopping, and recovery subroutines contained in a dedicated movement module.

The internal execution path of the MOVING state subactivity is detailed in figure 3 (Movement Logic). Because the FSM operates on a non-blocking, tick-based loop (`fsm_spin`), the movement logic evaluates a sequential chain of conditional checkpoints during each cycle:

1. Mid-Shaft Recovery: Evaluates if the elevator was previously halted between floors and determines the optimal recovery direction.
2. Floor Verification: A safety guard ensuring stop logic is not evaluated while the elevator is in transit between sensors, preventing invalid memory access.
3. Stop Evaluation: Checks the order queue against the current floor and travel direction to determine if a halt is required.
4. Directional Routing: Determines the next continuous direction of travel or returns the system to IDLE if the queue is empty.

The diagram uses the UML "End" icon to represent the early return behaviour, which ensures that the FSM yields control back to the main loop after resolving the movement logic for the current cycle.

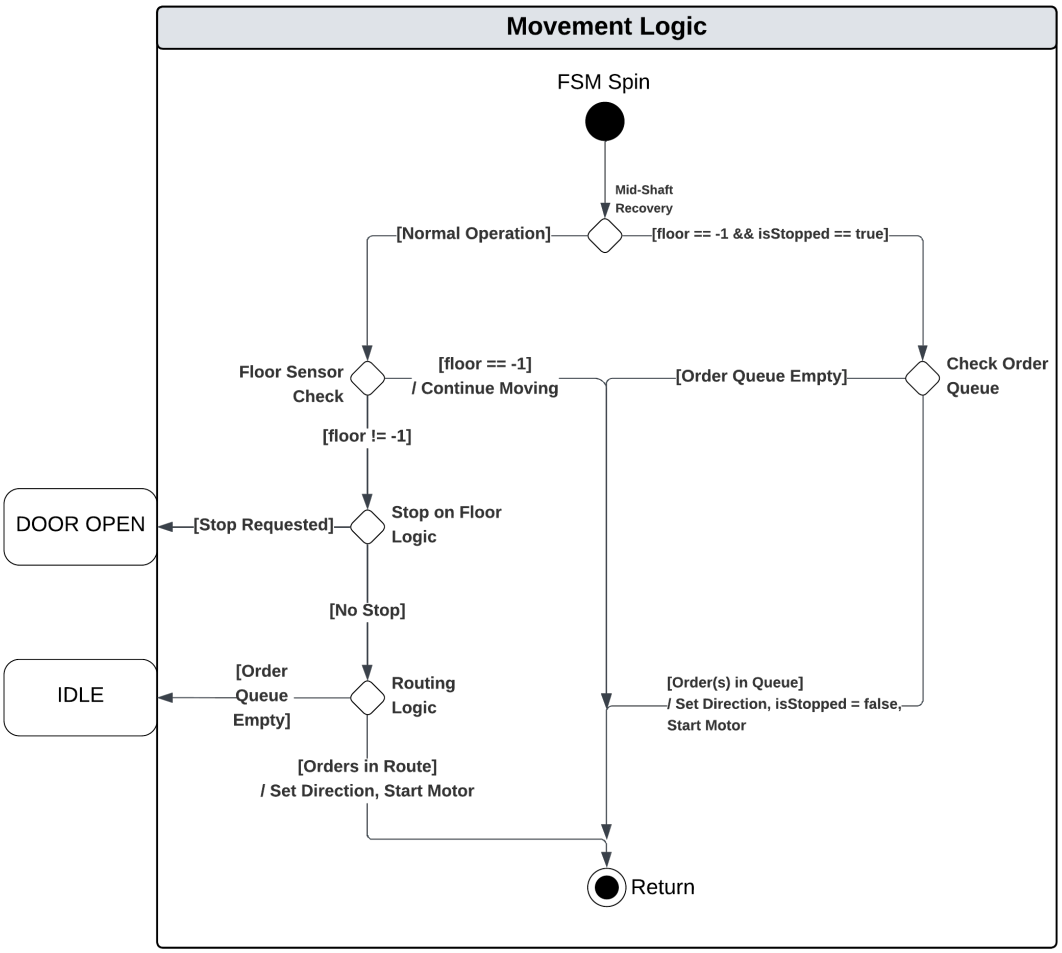

Figure 3: UML state machine diagram for the internal movement logic of the elevator system.

2.3 Software Implementation & Development

The system was implemented in C, utilizing an iterative development process (Fowler 2004, pp. 26–28). To maintain high code quality, the architecture was built modularly, allowing individual subsystems, such as the timer and routing logic, to be tested in isolation prior to full system integration. Throughout development, strict attention was paid to code readability, by using documentation and inline comments. Verification was conducted using a combination of unit testing for the routing algorithms using the provided simulator software and integration testing against the elevator hardware interface to ensure full compliance with the Product Requirements Specification (PRS).

2.3.1 Development Toolchain

The build process was automated using a `Makefile`, streamlining the compilation of the decoupled modules into a single executable. GNU Compiler Collection (GCC) was initially employed for the compilation of the executable. However, during the development phase, the `Makefile` was changed to compile using `Clang` against the `gnu11` standard to utilize some additional features, such as sanitization. To enforce strict memory safety and rigorous C programming practices, the build system was divided into two target modes. During active development, a `debug` mode was utilized with address and undefined behaviour sanitizers (`-fsanitize=address -fsanitize=undefined`) to systematically check for potential memory leaks and boundary violations. For the final compilation, a `release` mode was employed using maximum optimization (`-O3`) alongside a highly verbose set of warning flags (e.g., `-Wall -Wextra -Wpedantic -Wshadow`).

To satisfy the documentation requirements and maintain an Application Programming Interface (API) reference, Doxygen was integrated into the toolchain to automatically generate structural documentation directly from comments in the codebase, the results of which can be found on https://thebadmusician.github.io/TTK4235-elevator-project/.

2.3.2 Version Control & Continuous Integration

Source code management was handled via Git, with the repository hosted on GitHub. This version control approach was critical for maintaining the integrity of the codebase during the iterative development cycles, as well as tracking progress. Utilizing a remote repository allowed for seamless development from multiple locations, such as the Real-Time Lab, campus, and home office. Branching was used to isolate the development of different parts of the project (e.g., separating state machine logic updates from documentation).

Furthermore, to streamline documentation maintenance, a Continuous Integration (CI) pipeline was created using GitHub Actions. Upon every push to the remote repository, an automated workflow triggers to compile the latest Doxygen documentation via the project's `Doxyfile`. The action then automatically deploys the generated HyperText Markup Language (HTML) output to a live GitHub Pages site, which makes sure the system's API reference remains continuously updated and easily accessible without any manual overhead.

3 Results

The implemented elevator system successfully met all operational and safety requirements. During simulator integration testing, the core logic proved reliable. The movement module successfully prioritized orders in the current direction of travel and correctly bypassed hall calls in the opposite direction, strictly adhering to the routing logic specifications.

3.1 System Performance and Validation

Continuous execution and stress testing revealed a robust system architecture. The system successfully maintained the polling rate for hardware inputs and sensor readings without dropping orders during simultaneous button presses. Compilation and execution with address and undefined behavior sanitizers (`-fsanitize=address -fsanitize=undefined`) showed no memory leaks and zero boundary violations during operation. The system's state machine effectively handled all defined states and transitions, including edge cases such as mid-shaft emergency interruptions. The door control logic reliably managed the timing of door operations while maintaining responsiveness to user inputs.

3.2 Factory Acceptance Test

The system was evaluated against the Factory Acceptance Test (FAT) specifications defined in the PRD. A traceability matrix was utilized to map the specific PRSs to the observed system behaviour during integration testing, following the approach described in Laplante and Kassab 2023, Sections 3.5.7 and 5.6.2, albeit with a simplified and explicit design suitable for this project's scope. The results confirm that the system passes every specified test, detailed in tables 1 to 6.

Table 1: Verification of Startup & Initialization Requirements

Req ID	Req Description	Test Plan Reference	Result / Implementation
O1	Arrive at a defined state at startup.	FAT O1	System initializes motor downwards until a valid floor sensor is triggered.
O2	Ignore orders if in an undefined state.	FAT O2	`STATE_INIT` uses a blocking `while` loop, preventing order polling.
O3	Ignore unrealistic start conditions & buttons	FAT O3	`STATE_INIT` uses a blocking `while` loop, preventing button polling.

Table 2: Verification of Order Handling & Routing Logic

Req ID	Req Description	Test Plan Reference	Result / Implementation
H1	All orders shall be served eventually.	FAT H4, H5	Stress testing confirms no orders are dropped or cause the system to freeze.
H2	Ignore opposite direction orders while moving.	FAT H3	The routing algorithm bypasses UP hall calls when traveling DOWN (and vice versa).
H3	Dispatch all orders for a floor upon arrival.	FAT H1, H2	Both hall and cabin calls for the current floor are cleared upon door open.
H4	Stationary if no unserved orders.	FAT D5	FSM transitions to `STATE_IDLE` and halts motor when queue is empty.

Table 3: Verification of Hardware I/O and Indicators

Req ID	Req Description	Test Plan Reference	Result / Implementation
L1/L2	Order lights on when active, off when clear.	FAT L3, L4	Order matrix directly drives lamp states; cleared upon door closing after being opened.
L3/L5	Correct, singular floor light illuminated.	FAT L1	Floor indicator updates immediately upon sensor rising edge.
L4	Retain last floor light while moving.	FAT L2	Between-floor sensor reading (`-1`) does not overwrite last known floor lamp.
L6	Stop button illuminated only while pressed.	FAT S3	`hmi_stopBtnHandler` directly maps the button state to the lamp with debouncing logic to prevent rapid consecutive presses.

Table 4: Verification of Door Operations & Timers

Req ID	Req Description	Test Plan Reference	Result / Implementation
D1/D2	Open for 3s on arrival, then close.	FAT D1, D2, D4	Asynchronous-polling timer triggers door close and state transition after 3.0s timeout.
D3	Stop button at floor forces door open + 3s.	FAT S6	Door opens immediately if stopped at a valid floor; door open timer keeps resetting while the stop button is pressed.
D4	Obstruction keeps door open + 3s clear.	N/A	Door open timer keeps resetting while the obstruction switch is active.

Table 5: Verification of Safety Constraints & Emergency Stop

Req ID	Req Description	Test Plan Reference	Result / Implementation
S1/S2	Stationary when open / Only open at floor.	FAT D3	Motor explicitly halted in `STATE_DOOR_OPEN`. Door always remain closed mid-shaft, even if the stop button is pressed.
S4	Stop instantaneously on button press.	FAT S1	`STATE_STOP` halts motor right after transition within one 20ms spin cycle.
S5/S6	Delete orders / Ignore orders while held.	FAT S2, S4	Order matrix is wiped via `memset`. Button polling is ignored while stop button is active.
S7	Stationary upon release until new order.	FAT S5	After the door is closed (on floor only), elevator remains in `STATE_IDLE` waiting for orders.

Table 6: Verification of System Robustness & Recovery

Req ID	Req Description	Test Plan Reference	Result / Implementation
R1	Obstruction ignored if door closed.	N/A	Obstruction switch is only polled during `STATE_DOOR_OPEN`.
R2	No crashes, memory leaks, or restarts.	FAT R1	`-fsanitize=address` confirmed 0 leaks. System remained stable through FAT.
R3	Remember position / No recalibration.	FAT S6	`prevFloor` variable allows mid-shaft recovery using a recovery subroutine in movement module.

4 Discussion

4.1 V-model & Iterative Development Process

The application of the V-model and UML methodologies significantly influenced the project's lifecycle, especially by shifting the cognitive load from the implementation phase to the design phase. Creating the initial module and state machine diagrams beforehand provided a helpful oversight of the design requirements. It also assisted in identifying edge cases in advance, such as the system's behavior when interrupted between floors.

During the initial conceptual phase, the system architecture was drafted with four primary components: a FSM, a Timer, the provided elevio driver, and a Scheduler, as illustrated in figure 4. The intended purpose of the Scheduler was to manage the elevator routing, decision and movement logic, and the hardware button interaction.

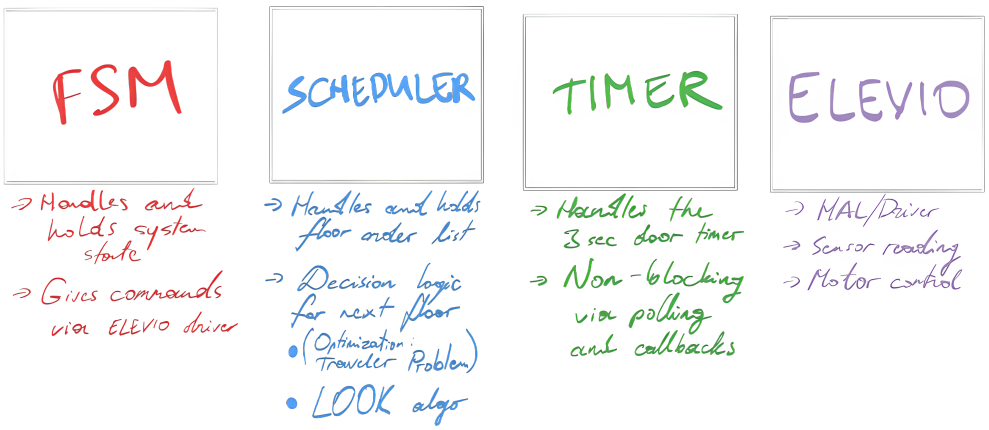

Figure 4: Initial software architecture concept with a Scheduler module.

However, attempting to map this concept into UML revealed that a centralized Scheduler would violate the principle of SOC by coupling multiple dependencies together. The initial concept would have made the system incredibly difficult to maintain and extend. The design was refactored before heavy implementation began, discarding the Scheduler and splitting the logic into the independent FSM, movement, and HMI modules, as shown in figure 1.

Throughout further phases of development, UML state machine diagrams (figure 2 and figure 3) served as an instruction set, ensuring that edge case handling was logically correct. In the final phase, the right side of the V-model (verification and validation) was realized through the traceability matrices and execution of FAT. Using the provided FAT test cases ensured that testing was requirement-driven, rather than impromptu code testing.

4.2 Robustness, Scalability, and Maintainability

While the robustness, scalability, and maintainability of the system were not explicitly focused on, the design choices made during development were influenced by these principles.

Modeling the system as an event-driven FSM severely restricted invalid operations. By defining strict transition conditions, the system became robust against unpredictable user inputs. The use of `Clang` sanitizers further verified this robustness at the lower level.

The use of a UML module dependency diagram ensured that the HAL was separate from the rest of the system. Because the driver is mostly isolated from the FSM, swapping the physical hardware or its API would only require rewriting the HMI module, leaving the core logic untouched.

Finally, while C is a procedural language, the module-based state machine design makes the system scalable. Adding a new feature, e.g., a "Fire Alarm" mode, would only require adding a new state

to the UML diagram and a corresponding handler function, without disrupting the existing routing algorithms.

4.3 AI Disclosure

Generative AI tools were utilized in a ”closed-loop” manner as an interactive *rubber duck* (Holland 2008) for brainstorming design, code review, debugging, and drafting Doxygen-style comments. This accelerated the documentation process, allowing more focus to be placed on the other parts of the project.

All of the output from the LLMs were read thoroughly and were critically analysed before being manually integrated to ensure that they did not contain any critical errors such as AI hallucinations.

5 Conclusion

The objective of this project was to design, implement, and verify a real-time elevator control system in C, satisfying the operational and safety requirements outlined in the PRD. Through simulator and hardware integration testing, the system successfully passed all FAT specifications, demonstrating reliable order routing and safe handling of edge cases, such as emergency interruptions.

The system’s architecture was guided by the V-model and UML methodologies during the initial design phase. By decomposing the software into decoupled modules, the design adhered to the principle of SOC, establishing a scalable codebase. Additionally, the use of memory sanitizers and an automated CI pipeline supported the development process, ensuring memory safety and providing consistent, up-to-date documentation.

Bibliography

Department of Engineering Cybernetics (2026). TTK4235 Heisprosjektet. TTK4235 Lab 2 - Heisprosjektet, NTNU, ITK. Provided via the course Blackboard page.

Fowler, Martin (2004). UML distilled: applying the standard object modeling language. eng. 3rd ed. The Addison-Wesley object technology series. Boston, MA: Addison-Wesley. ISBN: 9780134865126 9780134865133 9780321193681.

Holland, Ben (2008). Rubber Duck Debugging: Debugging software with a rubber ducky. URL: www.rubberduckdebugging.com (visited on 03/03/2026).

Laplante, Phillip A. and Mohamad Kassab (2023). What every engineer should know about software engineering. eng. Second edition. Boca Raton: CRC Press. ISBN: 9781032103181.

Appendix

A Product Requirements Document (PRD)

B FAT - Factory Acceptance Test

Kravspesifikasjonene som FAT-en er basert på finner dere i appendiks B.1, mens selve FAT-testen som baserer seg på kravspesifikasjonene finner dere i appendiks B.2. I praksis er det vanlig at kunde og leverandør avtaler disse på forhånd. FAT-en bestemmer i hvilken grad dere har implementert et korrekt system og er en direkte gjenspeiling av kravspesifikasjonen fra appendiks B.1, så om dere oppfyller alle kravene som er satt av den, har dere implementert et fullverdig system.

B.1 FAT - Heisspesifikasjoner

Krav: Oppstart

Punkt	Beskrivelse
O1	Ved oppstart skal heisen alltid komme til en definert tilstand. En definert tilstand betyr at styresystemet vet hvilken etasje heisen står i.
O2	Om heisen starter i en udefinert tilstand skal heissystemet ignorere alle forsøk på å gjøre bestillinger før systemet er kommet i en definert tilstand.
O3	Heissystemet skal ikke ta i betraktning urealistiske startbetingelser, som at heisen er over 4. etasje eller under 1. etasje, idet systemet skrus på.

Krav: Håndtering av bestillinger

Punkt	Beskrivelse
H1	Det skal ikke være mulig å komme i en situasjon hvor en bestilling ikke blir tatt. Alle bestillinger skal betjenes selv om nye bestillinger opprettes.
H2	Heisen skal ikke betjene bestillinger fra utenfor heisrommet dersom heisen er i bevegelse i motsatt retning av bestillingen.
H3	Når heisen først stopper i en etasje, skal det antas at alle som venter i etasjen går på, og at alle som skal av i etasjen går av. Dermed skal alle ordre i etasjen være regnet som ekspedert.
H4	Heisen skal stå stille dersom den ikke har noen ubetjente bestillinger.

Krav: Bestillings- og etasjelys

Punkt	Beskrivelse
L1	Når en bestilling gjøres, skal lyset i bestillingsknappen lyse helt til bestillingen er utført. Dette gjelder både bestillinger inne i heisen, og bestillinger utenfor.
L2	Om en bestillingsknapp ikke har en tilhørende bestilling, skal lyset i knappen være slukket.
L3	Når heisen er i en etasje skal korrekt etasjelys være tent.
L4	Når heisen er i bevegelse mellom to etasjer, skal etasjelyset til etasjen heisen sist var i være tent.
L5	Kun ett etasjelys skal være tent av gangen.
L6	Stoppknappen skal lyse så lenge denne er trykket inne. Den skal slukkes straks knappen slippes.

Krav: Heisdør

Punkt	Beskrivelse
D1	Når heisen ankommer en etasje det er gjort bestilling til, skal døren åpnes i 3 sekunder, for deretter å lukkes.
D2	Heisen skal være lukket når den ikke har ubetjente bestillinger.
D3	Hvis stoppknappen trykkes mens heisen er i en etasje, skal døren åpne seg. Døren skal forbli åpen så lenge stoppknappen er aktivert, og ytterligere 3 sekunder etter at stoppknappen er sluppet. Deretter skal døren lukke seg.
D4	Om obstruksjonsbryteren er aktivert mens døren først er åpen, skal den forbli åpen så lenge bryteren er aktiv. Når obstruksjonssignalet går lavt, skal døren lukke seg etter 3 sekunder.

Krav: Sikkerhet

Punkt	Beskrivelse
S1	Heisen skal alltid stå stille når døren er åpen.
S2	Heisdøren skal aldri åpne seg utenfor en etasje.
S3	Heisen skal aldri kjøre utenfor området definert av 1. til 4. etasje.
S4	Om stoppknappen trykkes, skal heisen stoppe momentant.
S5	Om stoppknappen trykkes, skal alle heisens ubetjente bestillinger slettes.
S6	Så lenge stoppknappen holdes inne, skal heisen ignorere alle forsøk på å gjøre bestillinger.
S7	Etter at stoppknappen er blitt sluppet, skal heisen stå i ro til den får nye bestillinger.

Krav: Robusthet

Punkt	Beskrivelse
R1	Obstruksjonsbryteren skal ikke påvirke systemet når døren ikke er åpen.
R2	Det skal ikke være nødvendig å starte programmet på nytt som følge av eksempelvis udefinert oppførsel som for eksempel at programmet krasjer, eller minnelekkasje.
R3	Etter at heisen først er kommet i en definert tilstand ved oppstart, skal ikke heisen trenge flere kalibreringsrunder for å vite hvor den er.

Krav: Tillegg

Punkt	Beskrivelse
Y1	Oppførsel som ikke er *vanlig heisoppførsel* kan gi trekk på FAT-testen. Når det er sagt så er det bare å bruke sunn fornuft og eventuelt spør vitass eller foreleser om noe er uklart.

B.2 FAT - Testspesifikasjoner

FAT-test: Oppstart

Punkt	Beskrivelse
O1	Sørger systemet for at heisen kommer i en definert tilstand?
O2	Ignoreres bestillinger før heisen har kommet i en definert tilstand?
O3	Ignoreres stoppknappen under initialisering?

FAT-test: Håndtering av bestillinger

Punkt	Beskrivelse
H1	Går heisen til riktig etasje når en bestilling mottas fra etasjepanelet?
H2	Går heisen til riktig etasje når en bestilling mottas fra heispanelet?
H3	Hvis heisen er på vei fra 4. etg til 1. etg og noen har bestilt OPP i 2. etg: kjører heisen til 1. etg før den kjører til 2. etg?
H4	Håndteres alle bestillingene hvis flere av bestillingsknappene trykkes samtidig?
H5	Vil alle bestillinger bli ekspedert, selv med vedvarende trykking av andre knapper (unntatt stopp), dvs. blir heisen aldri "fastlåst" mellom noen av etasjene?

FAT-test: Bestillingslys og etasjelys

Punkt	Beskrivelse
L1	Blir riktig etasjelys tent når heisen ankommer en etasje?
L2	Hvis heisen befinner seg mellom 2. og 3. etg og er på vei oppover, lyser etasjelyset i 2. etg?
L3	Blir lyset tent i bestillingsknappene når de blir trykket?
L4	Slukker lyset i bestillingsknappene når bestillingen er ekspedert, dvs. når heisen ankommer etasjen?

FAT-test: Heis-dør

Punkt	Beskrivelse
D1	Åpnes døren (lyser dørlyset) når heisen stopper i en etasje?
D2	Er døren åpen i 3 sekunder?
D3	Står heisen stille i de 3 sekundene døren er åpen?
D4	Lukkes døren før heisen kjøres videre?
D5	Lukkes døren og står heisen stille når det ikke er noen nye bestillinger?

FAT-test: Sikkerhet

Punkt	Beskrivelse
S1	Stopper heisen når stoppknappen trykkes?
S2	Blir bestillingene slettet (lysene på bestillingsknappene slukkes) når stoppknappen trykkes?
S3	Er lyset i stoppknappen tent mens stoppknappen er trykket?
S4	Ignoreres trykk på alle bestillingsknappene mens stoppknappen er trykket?
S5	Blir heisen stående i ro etter at stoppknappen er sluppet?
S6	Husker heisen hvor den er ved nødstopp mellom etasjer (dvs. kreves ikke ny initialisering)?
S7	Åpnes døren hvis stoppknappen aktiveres i en etasje?

FAT-test: Robusthet

Punkt	Beskrivelse
R1	Hvor stabilt er programmet? Må programmet startes på nytt under presentasjonen?